

Nama : Mohamad Zaky Bahtiar Arifianto
Matkul : Praktikum Pemrograman Berorientasi Obyek
Jurusan : Teknik Informatika
NRP : 3125522015

Tugas Pertemuan 3

BAB I: TUJUAN PRAKTIKUM

Berdasarkan Buku Panduan Praktikum Pemrograman Berorientasi Obyek Modul 3, tujuan instruksional yang harus dicapai oleh mahasiswa adalah:

- Mampu menjelaskan tujuan fundamental encapsulation dan information hiding dalam rekayasa perangkat lunak berorientasi objek.
- Mampu menggunakan access modifier private, public, protected, dan default secara tepat dan proporsional.
- Mampu membatasi akses langsung terhadap attribute kelas untuk mencegah modifikasi data yang tidak sah.
- Mampu membuat method getter (accessor) dan setter (mutator) yang terstandarisasi.
- Mampu menambahkan mekanisme validasi data ketat melalui method setter dan constructor.
- Mampu melakukan proses refactoring menyeluruh pada class hasil praktikum sebelumnya (P2).
- Mampu melaksanakan tahapan Agile Sprint sederhana serta pengujian fungsional data valid dan invalid.

BAB II: LANDASAN TEORI & KONSEP DASAR ENCAPSULATION

2.1 Konsep Encapsulation & Information Hiding

Encapsulation (pengkapsulan) adalah pilar utama pemrograman berorientasi objek yang membungkus data (atribut) dan perilaku (method) ke dalam satu unit tunggal (class), sekaligus menyembunyikan rincian implementasi internal dari akses langsung pihak luar (information hiding). Dengan prinsip ini, data objek tidak dapat diintervensi secara langsung tanpa melalui antarmuka resmi yang telah ditentukan.

2.2 Tingkatan Access Modifier

Bahasa pemrograman Java menyediakan 4 jenis access modifier untuk mengontrol visibilitas komponen:

- 1. private: Komponen hanya dapat diakses dari dalam tubuh class yang sama. Ini merupakan modifier wajib bagi atribut inti dalam prinsip enkapsulasi.
- 2. default (package-private): Komponen dapat diakses oleh class-class lain yang berada dalam package yang sama.
- 3. protected: Komponen dapat diakses dalam package yang sama dan oleh subclass (turunan) meskipun berada di luar package.
- 4. public: Komponen dapat diakses secara bebas dari bagian program mana pun.

2.3 Peran Getter dan Setter Selektif

Getter adalah method publik yang berfungsi untuk membaca nilai atribut tanpa memberikan izin modifikasi. Sementara itu, Setter adalah method publik yang digunakan untuk mengubah nilai atribut.

Dalam perancangan OOP yang baik, setter tidak dibuat untuk semua atribut. Atribut yang bersifat pengenalan tetap (seperti NRP atau Kode Mata Kuliah) cukup diberikan getter saja agar nilainya tetap aman (immutable).

2.4 Validasi Data dan Refactoring Constructor

Keunggulan utama setter terletak pada kemampuannya menyaring data melalui logika validasi (misal mengecek rentang numerik atau string kosong). Agar constructor tidak mengabaikan aturan bisnis tersebut, constructor harus direfactor untuk memanfaatkan logika validasi atau memanggil setter saat inialisasi awal objek.

BAB III: REVIEW P2 & SPRINT PLANNING P3

3.1 Evaluasi Keterbatasan Praktikum P2

Pada praktikum P2, seluruh atribut dideklarasikan public sehingga rentan terhadap kesalahan mutasi state. Sebagai contoh, sebuah objek Mahasiswa dapat diberikan nilai IPK -10.0 atau semester 0 dari class Main tanpa terdeteksi error. Kelemahan inilah yang diatasi secara tuntas pada Sprint P3 melalui refactoring enkapsulasi.

3.2 Agile Sprint 3 (P3)

- Sprint Goal P3: "Memperbaiki struktur class proyek SIAKAD (Mahasiswa, MataKuliah, dan KRS) dengan menerapkan encapsulation penuh sehingga data object hanya dapat diakses dan diubah melalui mekanisme yang terkontrol, menerapkan minimal 3 aturan validasi ketat, serta memverifikasi ketahanan sistem melalui pengujian data valid dan data invalid pada Main.java."

Tabel Sprint Backlog Modul 3:

ID	Item Sprint Backlog	Estimasi Bobot	Status Akhir
SB-01	Melakukan audit struktur class P2 dan pemetaan perbaikan enkapsulasi	15%	DONE
SB-02	Mengubah seluruh atribut utama menjadi private pada 3 class	25%	DONE
SB-03	Membuat method getter untuk atribut yang perlu dibaca	15%	DONE
SB-04	Membuat setter selektif dengan aturan validasi data ketat	20%	DONE
SB-05	Memperbaiki constructor agar memanfaatkan validasi	10%	DONE

data

SB-06	Mengimplementasikan pengujian Test Valid & Test Invalid pada Main	10%	DONE
SB-07	Menyusun laporan resmi Word, PDF eksekutif, dan dokumentasi README	5%	DONE

BAB IV: AUDIT CLASS P2 & PEMETAAN ENCAPSULATION P3

4.1 Tabel Audit Class P2

Identifikasi kondisi atribut sebelum dan sesudah refactoring Modul 3:

Class	Attribute	Kondisi P2	Perbaikan P3 & Aturan Validasi
Mahasiswa	nrp	public	private + getter only (identitas unik, immutable).
Mahasiswa	nama, prodi	public	private + getter + setter (validasi tidak null/blank, min 3 huruf).
Mahasiswa	semester, ipk	public	private + getter + setter (validasi semester 1-14, IPK 0.00-4.00).
Mahasiswa	totalSks	public	private + getter only (hanya dimutasi via method transaksi).
MataKuliah	kodeMk, sks	public	private + getter only (standar kurikulum tetap, immutable).
MataKuliah	namaMk, dosen, kuota	public	private + getter + setter (validasi kuota > 0 & >= peserta).
KRS	nomorKrs, mhs, daftarMk	public	private + getter (hanya dimanipulasi via method tambahMK).

4.2 Tabel Spesifikasi Encapsulation P3 (Modul 3 Halaman 2)

Pemetaan lengkap class, private attribute, getter, setter, dan mekanisme validasinya:

Class	Private Attribute	Getter	Setter	Validation Rules
Mahasiswa	nrp, nama, prodi, semester, ipk, totalSks	getNrp(), getNama(), getProdi(), getSemester(), getIpk(), getTotalSks()	setNama(), setProdi(), setSemester(), setIpk()	• nama: min 3 char, not blank • semester: 1 s/d 14 • ipk: 0.00 s/d 4.00 • sks: <= kuota IPK
MataKuliah	kodeMk, namaMk, sks, semBuka, dosen, kuota, peserta	getKodeMk(), getNamaMk(), getSks(), getSem(), getDosen(), getKuota(), getPeserta()	setNamaMk(), setSem(), setDosen(), setKuotaKelas()	• namaMk & dosen: not blank • semBuka: 1 s/d 8 • kuota: > 0 & >= pesertaTerdaftar
KRS	nomorKrs, mhs, thnAjaran, semKrs, daftarMk[], jumlahMk, status, dosenWali	getNomorKrs(), getMhs(), getThn(), getSem(), getDaftarMk(), getJumlah(), isDisetujui()	setTahunAjaran(), setSemesterKrs()	• mhs != null • krs tidak boleh kosong saat disahkan • kuota kelas & batas SKS tervalidasi

BAB V: IMPLEMENTASI SOURCE CODE JAVA

Berikut adalah source code Java yang telah direfactor dan ditempatkan pada folder src/:

5.1 Source Code: src/Mahasiswa.java

```
/**
 * Class Mahasiswa merepresentasikan entitas mahasiswa dalam Sistem Informasi Akademik (SIKAD).
 * Menerapkan prinsip Encapsulation penuh dengan access modifier private pada semua atribut,
 * menyediakan getter dan setter selektif dengan validasi data ketat, serta constructor
 * tervalidasi.
 */
*
* Praktikum Pemrograman Berorientasi Obyek (PBO) - Modul 3
* Pengembang: Mohamad Zaky Bahtiar Arifianto (NRP: 3125522015)
* Institusi : Politeknik Elektronika Negeri Surabaya (PENS PSDKU Sumenep)
*/
public class Mahasiswa {
    // =====
    // 1. ATTRIBUTES (PRIVATE - ENCAPSULATION & INFORMATION HIDING)
    // =====
    private String nrp;
    private String nama;
    private String prodi;
    private int semester;
    private double ipk;
    private int totalSks;

    // =====
    // 2. CONSTRUCTOR (TERVALIDASI)
    // =====
}
/**
```

```

    * Constructor tervalidasi: inisialisasi data pokok melalui setter
    * untuk memastikan tidak ada input data yang melanggar aturan bisnis sejak awal.
    */
    public Mahasiswa(String nrp, String nama, String prodi, int semester, double ipk) {
        setNrpInternal(nrp);
        setName(nama);
        setProdi(prodi);
        setSemester(semester);
        setIpk(ipk);
        this.totalSks = 0;
    }

    // =====
    // 3. GETTER (ACCESSOR METHODS)
    // =====
    public String getNrp() {
        return this.nrp;
    }

    public String getName() {
        return this.nama;
    }

    public String getProdi() {
        return this.prodi;
    }

    public int getSemester() {
        return this.semester;
    }

    public double getIpk() {
        return this.ipk;
    }

    public int getTotalSks() {
        return this.totalSks;
    }

    // =====
    // 4. SETTER DENGAN VALIDASI KETAT (MUTATOR METHODS)
    // =====
    /**
     * Validasi internal NRP saat instansiasi (NRP bersifat unik dan read-only setelah dibuat).
     */
    private void setNrpInternal(String nrp) {
        if (nrp != null && !nrp.trim().isEmpty()) {
            this.nrp = nrp.trim();
        } else {
            this.nrp = "0000000000";
            System.out.println("[VALIDASI ERROR] NRP tidak boleh kosong/null! Diset ke default.");
        }
    }

    /**
     * Validasi Nama: tidak boleh null, tidak boleh kosong/whitespace, minimal 3 karakter.
     */
    public void setName(String nama) {
        if (nama != null && nama.trim().length() >= 3) {

```

```

        this.nama = nama.trim();
    } else {
        System.out.println("[VALIDASI DITOLAK] Nama mahasiswa tidak valid! Minimal 3
karakter dan tidak boleh kosong.");
    }
}

/**
 * Validasi Program Studi: tidak boleh null atau kosong.
 */
public void setProdi(String prodi) {
    if (prodi != null && !prodi.trim().isEmpty()) {
        this.prodi = prodi.trim();
    } else {
        System.out.println("[VALIDASI DITOLAK] Program studi tidak valid! Nilai tidak boleh
kosong.");
    }
}

/**
 * Validasi Semester: harus berada pada rentang batas akademik perkuliahan (1 s/d 14).
 */
public void setSemester(int semester) {
    if (semester >= 1 && semester <= 14) {
        this.semester = semester;
    } else {
        System.out.println("[VALIDASI DITOLAK] Semester " + semester + " tidak valid! Batas
semester adalah 1 s/d 14.");
    }
}

/**
 * Validasi Capaian IPK: harus berada pada rentang 0.00 s/d 4.00.
 */
public void setIpk(double ipk) {
    if (ipk >= 0.0 && ipk <= 4.0) {
        this.ipk = ipk;
    } else {
        System.out.printf("[VALIDASI DITOLAK] Nilai IPK %.2f tidak valid! Harus berada di
rentang 0.00 s/d 4.00.\n", ipk);
    }
}

// =====
// 5. BUSINESS LOGIC METHODS
// =====
/**
 * Menghitung kuota batas beban SKS semester aktif berdasarkan regulasi akademik.
 */
public int hitungBebanMaksimalSks() {
    if (this.ipk >= 3.00) {
        return 24;
    } else if (this.ipk >= 2.50) {
        return 21;
    } else if (this.ipk >= 2.00) {
        return 18;
    } else {
        return 15;
    }
}
}

```

```

/**
 * Menambahkan akumulasi beban SKS dengan pengecekan batas maksimal.
 */
public boolean tambahSks(int sks) {
    int batasMaks = this.hitungBebanMaksimalSks();
    if (sks > 0 && (this.totalSks + sks <= batasMaks)) {
        this.totalSks += sks;
        return true;
    } else {
        return false;
    }
}

/**
 * Mereset total SKS kembali ke 0.
 */
public void resetSks() {
    this.totalSks = 0;
    System.out.println("[INFO MAHASISWA] Total SKS untuk " + this.nama + " direset ke 0.");
}

/**
 * Menampilkan profil lengkap mahasiswa.
 */
public void tampilkanProfil() {
    System.out.println("-----");
    System.out.println("                PROFIL AKADEMIK MAHASISWA                ");
    System.out.println("-----");
    System.out.println("NRP                : " + this.nrp);
    System.out.println("Nama Lengkap       : " + this.nama);
    System.out.println("Program Studi      : " + this.prodi);
    System.out.println("Semester           : " + this.semester);
    System.out.printf("IPK Kumulatif      : %.2f\n", this.ipk);
    System.out.println("Batas Maksimal SKS: " + this.hitungBebanMaksimalSks() + " SKS");
    System.out.println("Total SKS Diambil  : " + this.totalSks + " SKS");
    System.out.println("Status KRS         : " + (this.totalSks > 0 ? "Aktif Terisi" : "Belum Mengisi"));
    System.out.println("-----");
}
}

```

5.2 Source Code: src/MataKuliah.java

```

/**
 * Class MataKuliah merepresentasikan entitas mata kuliah kurikulum akademik.
 * Menerapkan prinsip Encapsulation dengan seluruh atribut berstatus private,
 * serta dilengkapi getter dan setter selektif dengan validasi kapasitas dan integritas data.
 *
 * Praktikum Pemrograman Berorientasi Obyek (PBO) - Modul 3
 * Pengembang: Mohamad Zaky Bahtiar Arifianto (NRP: 3125522015)
 * Institusi : Politeknik Elektronika Negeri Surabaya (PENS PSDKU Sumenep)
 */
public class MataKuliah {
    // =====
    // 1. ATTRIBUTES (PRIVATE - ENCAPSULATION)
    // =====

```

```

private String kodeMk;
private String namaMk;
private int sks;
private int semesterBuka;
private String dosenPengampu;
private int kuotaKelas;
private int pesertaTerdaftar;

// =====
// 2. CONSTRUCTOR (TERVALIDASI)
// =====
public MataKuliah(String kodeMk, String namaMk, int sks, int semesterBuka, String
dosenPengampu, int kuotaKelas) {
    setKodeMkInternal(kodeMk);
    setNamaMk(namaMk);
    setSksInternal(sks);
    setSemesterBuka(semesterBuka);
    setDosenPengampu(dosenPengampu);
    this.pesertaTerdaftar = 0;
    setKuotaKelas(kuotaKelas);
}

// =====
// 3. GETTER (ACCESSOR METHODS)
// =====
public String getKodeMk() {
    return this.kodeMk;
}

public String getNamaMk() {
    return this.namaMk;
}

public int getSks() {
    return this.sks;
}

public int getSemesterBuka() {
    return this.semesterBuka;
}

public String getDosenPengampu() {
    return this.dosenPengampu;
}

public int getKuotaKelas() {
    return this.kuotaKelas;
}

public int getPesertaTerdaftar() {
    return this.pesertaTerdaftar;
}

// =====
// 4. SETTER DENGAN VALIDASI KETAT (MUTATOR METHODS)
// =====
private void setKodeMkInternal(String kodeMk) {
    if (kodeMk != null && !kodeMk.trim().isEmpty()) {
        this.kodeMk = kodeMk.trim();
    } else {

```



```

        this.kodeMk = "MK000";
        System.out.println("[VALIDASI ERROR] Kode MK tidak boleh kosong! Diset ke MK000.");
    }
}

private void setSksInternal(int sks) {
    if (sks >= 1 && sks <= 6) {
        this.sks = sks;
    } else {
        this.sks = 2; // Default fallback SKS
        System.out.println("[VALIDASI ERROR] Bobot SKS " + sks + " tidak valid (harus 1-6)! Diset ke default 2 SKS.");
    }
}

public void setNamaMk(String namaMk) {
    if (namaMk != null && !namaMk.trim().isEmpty()) {
        this.namaMk = namaMk.trim();
    } else {
        System.out.println("[VALIDASI DITOLAK] Nama mata kuliah tidak boleh kosong!");
    }
}

public void setSemesterBuka(int semesterBuka) {
    if (semesterBuka >= 1 && semesterBuka <= 8) {
        this.semesterBuka = semesterBuka;
    } else {
        System.out.println("[VALIDASI DITOLAK] Semester buka " + semesterBuka + " tidak valid! Harus rentang 1 s/d 8.");
    }
}

public void setDosenPengampu(String dosenPengampu) {
    if (dosenPengampu != null && dosenPengampu.trim().length() >= 3) {
        this.dosenPengampu = dosenPengampu.trim();
    } else {
        System.out.println("[VALIDASI DITOLAK] Nama dosen pengampu tidak valid! Minimal 3 karakter.");
    }
}

/**
 * Validasi Kuota Kelas: harus positif (> 0) dan tidak boleh lebih kecil dari peserta yang sudah terdaftar.
 */
public void setKuotaKelas(int kuotaKelas) {
    if (kuotaKelas <= 0) {
        System.out.println("[VALIDASI DITOLAK] Kuota kelas " + kuotaKelas + " tidak valid! Kuota harus lebih besar dari 0.");
    } else if (kuotaKelas < this.pesertaTerdaftar) {
        System.out.println("[VALIDASI DITOLAK] Kuota baru (" + kuotaKelas + ") tidak boleh lebih kecil dari jumlah mahasiswa yang sudah terdaftar (" + this.pesertaTerdaftar + ")!");
    } else {
        this.kuotaKelas = kuotaKelas;
    }
}

// =====

```

```

// 5. BUSINESS LOGIC METHODS
// =====
public boolean tambahPeserta() {
    if (!isKelasPenuh()) {
        this.pesertaTerdaftar++;
        return true;
    } else {
        return false;
    }
}

public int getSisaKuota() {
    return this.kuotaKelas - this.pesertaTerdaftar;
}

public boolean isKelasPenuh() {
    return this.pesertaTerdaftar >= this.kuotaKelas;
}

public void tampilkanDetailMk() {
    System.out.println("-----");
    System.out.println("                        INFORMASI MATA KULIAH                        ");
    System.out.println("-----");
    System.out.println("Kode MK           : " + this.kodeMk);
    System.out.println("Nama Mata Kuliah  : " + this.namaMk);
    System.out.println("Bobot SKS         : " + this.sks + " SKS");
    System.out.println("Semester Buka     : Semester " + this.semesterBuka);
    System.out.println("Dosen Pengampu    : " + this.dosenPengampu);
    System.out.println("Kapasitas Kelas   : " + this.pesertaTerdaftar + " / " +
this.kuotaKelas + " Mahasiswa");
    System.out.println("Sisa Kuota Kursi   : " + this.getSisaKuota() + " Kursi");
    System.out.println("Status Ketersediaan: " + (this.isKelasPenuh() ? "[PENUH]" :
"[TERSEDIA]"));
    System.out.println("-----");
}
}

```

5.3 Source Code: src/KRS.java

```

/**
 * Class KRS (Kartu Rencana Studi) merepresentasikan dokumen rencana studi semester mahasiswa.
 * Menerapkan prinsip Encapsulation dengan membungkus agregasi Mahasiswa dan array MataKuliah,
 * menjamin validasi aturan beban SKS, ketersediaan kuota kelas, serta persetujuan resmi Dosen
 * Wali.
 *
 * *
 * Praktikum Pemrograman Berorientasi Obyek (PBO) - Modul 3
 * Pengembang: Mohamad Zaky Bahtiar Arifianto (NRP: 3125522015)
 * Institusi : Politeknik Elektronika Negeri Surabaya (PENS PSDKU Sumenep)
 */
public class KRS {
    // =====
    // 1. ATTRIBUTES (PRIVATE - ENCAPSULATION)
    // =====
    private String nomorKrs;
    private Mahasiswa mahasiswa;
    private String tahunAjaran;
    private int semesterKrs;
}

```

```

private MataKuliah[] daftarMataKuliah;
private int jumlahMk;
private boolean statusValidasi;
private String dosenWaliPengesah;

// =====
// 2. CONSTRUCTOR (TERVALIDASI)
// =====
public KRS(String nomorKrs, Mahasiswa mahasiswa, String tahunAjaran, int semesterKrs, int
kapasitasMaksMk) {
    if (nomorKrs != null && !nomorKrs.trim().isEmpty()) {
        this.nomorKrs = nomorKrs.trim();
    } else {
        this.nomorKrs = "KRS-DEFAULT";
    }

    if (mahasiswa != null) {
        this.mahasiswa = mahasiswa;
    } else {
        System.out.println("[FATAL ERROR] Objek mahasiswa tidak boleh null saat inisialisasi
KRS!");
    }

    setTahunAjaran(tahunAjaran);
    setSemesterKrs(semesterKrs);

    int kapasitas = kapasitasMaksMk > 0 ? kapasitasMaksMk : 10;
    this.daftarMataKuliah = new MataKuliah[kapasitas];
    this.jumlahMk = 0;
    this.statusValidasi = false;
    this.dosenWaliPengesah = "-";
}

// =====
// 3. GETTER (ACCESSOR METHODS)
// =====
public String getNomorKrs() {
    return this.nomorKrs;
}

public Mahasiswa getMahasiswa() {
    return this.mahasiswa;
}

public String getTahunAjaran() {
    return this.tahunAjaran;
}

public int getSemesterKrs() {
    return this.semesterKrs;
}

public MataKuliah[] getDaftarMataKuliah() {
    return this.daftarMataKuliah;
}

public int getJumlahMk() {
    return this.jumlahMk;
}

```

```

public boolean isDisetujui() {
    return this.statusValidasi;
}

public String getDosenWaliPengesah() {
    return this.dosenWaliPengesah;
}

// =====
// 4. SETTER DENGAN VALIDASI KETAT (MUTATOR METHODS)
// =====
public void setTahunAjaran(String tahunAjaran) {
    if (tahunAjaran != null && !tahunAjaran.trim().isEmpty()) {
        this.tahunAjaran = tahunAjaran.trim();
    } else {
        System.out.println("[VALIDASI DITOLAK] Tahun ajaran tidak boleh kosong!");
    }
}

public void setSemesterKrs(int semesterKrs) {
    if (semesterKrs >= 1 && semesterKrs <= 14) {
        this.semesterKrs = semesterKrs;
    } else {
        System.out.println("[VALIDASI DITOLAK] Semester KRS tidak valid (harus 1 s/d 14)!");
    }
}

// =====
// 5. BUSINESS LOGIC METHODS
// =====
public boolean tambahMataKuliah(MataKuliah mk) {
    if (mk == null) {
        System.out.println("    [GAGAL] Objek mata kuliah null!");
        return false;
    }

    System.out.println(">> [PERMOHONAN KRS] Mahasiswa " + this.mahasiswa.getNama() +
        " mendaftar MK: " + mk.getNamaMk() + " (" + mk.getSks() + " SKS)...");

    if (this.jumlahMk >= this.daftarMataKuliah.length) {
        System.out.println("    [GAGAL] Kapasitas maksimum item kartu rencana studi penuh!");
        return false;
    }

    if (mk.isKelasPenuh()) {
        System.out.println("    [GAGAL] Kelas mata kuliah '" + mk.getNamaMk() + "' sudah penuh
(" + mk.getPesertaTerdaftar() + "/" + mk.getKuotaKelas() + " Kursi)!");
        return false;
    }

    int batasSks = this.mahasiswa.hitungBebanMaksimalSks();
    int totalSksSekarang = this.hitungTotalSksKrs();
    if (totalSksSekarang + mk.getSks() > batasSks) {
        System.out.println("    [GAGAL] Melebihi batas kuota SKS mahasiswa (" + batasSks + "
SKS)! " +
        "Total saat ini: " + totalSksSekarang + " SKS + MK: " + mk.getSks()
+ " SKS.");
        return false;
    }
}

```

```

        this.daftarMataKuliah[this.jumlahMk] = mk;
        this.jumlahMk++;
        mk.tambahPeserta();
        this.mahasiswa.tambahSks(mk.getSks());

        System.out.println("    [BERHASIL] MK '" + mk.getNamaMk() + "' berhasil ditambahkan ke KRS "
+ this.nomorKrs);
        System.out.println("    Beban SKS saat ini: " + this.hitungTotalSksKrs() + " / " + batasSks
+ " SKS");
        return true;
    }

    public void setujuKrs(String namaDosenWali) {
        if (this.jumlahMk == 0) {
            System.out.println("[PERINGATAN] KRS " + this.nomorKrs + " belum memiliki mata kuliah,
tidak dapat disahkan.");
            return;
        }
        if (namaDosenWali == null || namaDosenWali.trim().isEmpty()) {
            System.out.println("[VALIDASI DITOLAK] Nama dosen wali pengesah tidak boleh kosong!");
            return;
        }
        this.statusValidasi = true;
        this.dosenWaliPengesah = namaDosenWali.trim();
        System.out.println("[VALIDASI KRS] Dokumen KRS " + this.nomorKrs + " milik " +
this.mahasiswa.getNama() +
            " berhasil DISETUJUI & DISAHKAN oleh Dosen Wali: " +
this.dosenWaliPengesah);
    }

    public int hitungTotalSksKrs() {
        int total = 0;
        for (int i = 0; i < this.jumlahMk; i++) {
            total += this.daftarMataKuliah[i].getSks();
        }
        return total;
    }

    public void tampilkanKrs() {
System.out.println("=====");
        System.out.println("                                KARTU RENCANA STUDI (KRS)
");
        System.out.println("                                POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
");
System.out.println("=====");
        System.out.println("Nomor Registrasi KRS : " + this.nomorKrs);
        System.out.println("Tahun Ajaran / Sem. : " + this.tahunAjaran + " (Semester " +
this.semesterKrs + ")");
        System.out.println("Mahasiswa (NRP>Nama) : " + this.mahasiswa.getNrp() + " - " +
this.mahasiswa.getNama());
        System.out.println("Program Studi : " + this.mahasiswa.getProdi());
        System.out.printf("Capaian IPK Lalu : %.2f (Batas Maksimal Beban: %d SKS)\n",
            this.mahasiswa.getIpk(), this.mahasiswa.hitungBebanMaksimalSks());
        System.out.println("Status Pengesahan : " + (this.statusValidasi ? "DISETUJUI (VALID)" :
"PENDING (BELUM DISETUJUI)");
        System.out.println("Dosen Wali Pengesah : " + this.dosenWaliPengesah);
        System.out.println("-----");
    }

```

```

---");
    System.out.printf("%-4s | %-10s | %-30s | %-5s | %-20s\n", "NO", "KODE", "MATA KULIAH",
"SKS", "DOSEN PENGAMPU");
    System.out.println("-----");
---");
    if (this.jumlahMk == 0) {
        System.out.println("                [ Belum ada mata kuliah yang diambil ]
");
    } else {
        for (int i = 0; i < this.jumlahMk; i++) {
            MataKuliah mk = this.daftarMataKuliah[i];
            System.out.printf("%-4d | %-10s | %-30s | %-5d | %-20s\n",
                (i + 1), mk.getKodeMk(), mk.getNamaMk(), mk.getSks(),
mk.getDosenPengampu());
        }
    }
    System.out.println("-----");
---");
    System.out.printf("TOTAL SKS TERDAFTAR : %d SKS\n", this.hitungTotalSksKrs());

System.out.println("=====\n");
}
}

```

5.4 Source Code: src/Main.java

```

/**
 * Class Main merupakan program pengujian utama Praktikum PBO Modul 3.
 * Menguji implementasi Encapsulation, Access Modifier private, Getter-Setter,
 * serta verifikasi ketat melalui dua skenario utama:
 * 1. TEST VALID: Instansiasi objek dan modifikasi data yang sah via setter.
 * 2. TEST INVALID: Uji ketahanan validasi data saat diberi input salah/ilegal.
 *
 * Praktikum Pemrograman Berorientasi Obyek (PBO) - Modul 3
 * Pengembang: Mohamad Zaky Bahtiar Arifianto (NRP: 3125522015)
 * Institusi : Politeknik Elektronika Negeri Surabaya (PENS PSDKU Sumenep)
 */
public class Main {
    public static void main(String[] args) {

System.out.println("=====");
        System.out.println("        PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK (PBO) - MODUL 3
");
        System.out.println("        Encapsulation, Access Modifier, Getter-Setter, dan Validasi Data
");

System.out.println("=====");
        System.out.println("Pengembang    : Mohamad Zaky Bahtiar Arifianto");
        System.out.println("NRP          : 3125522015");
        System.out.println("Program Studi: D3 Teknik Informatika");
        System.out.println("Institusi    : PENS PSDKU Sumenep\n");

        // =====
        // BAGIAN 1: PENGUJIAN DATA VALID (TEST VALID)
        // =====

System.out.println("#####");

```

```

        System.out.println("                                [SKENARIO 1: PENGUJIAN DATA VALID (TEST VALID)]
");
System.out.println("#####");

        System.out.println("\n>>> 1.1 INSTANSIASI OBJEK DENGAN CONSTRUCTOR TERVALIDASI");
        Mahasiswa mhs1 = new Mahasiswa("3125522015", "Mohamad Zaky Bahtiar Arifianto", "D3 Teknik
Informatika", 3, 3.82);
        Mahasiswa mhs2 = new Mahasiswa("3125522022", "Ahmad Wildan Prasetyo", "D3 Teknik
Informatika", 3, 2.45);

        MataKuliah mk1 = new MataKuliah("IF201", "Pemrograman Berorientasi Obyek", 3, 3, "Nirwana
Haidar Hari, S.Pd., M.Kom.", 30);
        MataKuliah mk2 = new MataKuliah("IF202", "Praktikum PBO", 2, 3, "Nirwana Haidar Hari,
S.Pd., M.Kom.", 30);
        MataKuliah mk3 = new MataKuliah("IF203", "Basis Data Lanjut", 3, 3, "Firman Arifin, S.T.,
M.T.", 2);
        MataKuliah mk4 = new MataKuliah("IF204", "Rekayasa Perangkat Lunak", 3, 3, "Budi Raharjo,
S.Kom., M.Kom.", 25);
        MataKuliah mk5 = new MataKuliah("IF205", "Jaringan Komputer", 3, 3, "Hendra Kusuma, S.Kom.,
M.T.", 30);

        KRS krs1 = new KRS("KRS-2026-001", mhs1, "2026/2027 Ganjil", 3, 8);
        KRS krs2 = new KRS("KRS-2026-002", mhs2, "2026/2027 Ganjil", 3, 8);
        System.out.println("[STATUS TEST VALID] Seluruh objek berhasil dibuat melalui
constructor.");

        System.out.println("\n>>> 1.2 MEMBACA DATA MENGGUNAKAN GETTER (ACCESSOR)");
        System.out.println("Mhs1 - NRP (getNrp)      : " + mhs1.getNrp());
        System.out.println("Mhs1 - Nama (getNama)     : " + mhs1.getNama());
        System.out.printf("Mhs1 - IPK (getIpk)        : %.2f\n", mhs1.getIpk());
        System.out.println("Mhs1 - Kuota SKS (hitung): " + mhs1.hitungBebanMaksimalSks() + " SKS");
        System.out.println("MK1 - Nama MK (getNama)  : " + mk1.getNamaMk() + " (" + mk1.getSks() +
" SKS)");
        System.out.println("MK1 - Dosen (getDosen)   : " + mk1.getDosenPengampu());

        System.out.println("\n>>> 1.3 MENGUBAH DATA MENGGUNAKAN SETTER VALID (MUTATOR)");
        mhs1.setIpk(3.92);
        System.out.printf("[HASIL SETTER VALID] IPK mhs1 setelah setIpk(3.92): %.2f\n",
mhs1.getIpk());
        mk3.setDosenPengampu("Dr. Indah Susilowati, S.T., M.T.");
        System.out.println("[HASIL SETTER VALID] Dosen MK3 setelah setDosenPengampu: " +
mk3.getDosenPengampu());

        System.out.println("\n>>> 1.4 TRANSAKSI PENGISIAN KRS & PENGESAHAN OLEH DOSEN WALI");
        krs1.tambahMataKuliah(mk1);
        krs1.tambahMataKuliah(mk2);
        krs1.tambahMataKuliah(mk3);
        krs1.tambahMataKuliah(mk4);
        krs1.tambahMataKuliah(mk5);
        krs1.setujuiKrs("Nirwana Haidar Hari, S.Pd., M.Kom.");
        System.out.println("Status Verifikasi KRS 1 (isDisetujui): " + (krs1.isDisetujui() ?
"VALID" : "PENDING"));

        // =====
        // BAGIAN 2: PENGUJIAN DATA INVALID (TEST INVALID - ATURAN VALIDASI)
        // =====

System.out.println("\n#####");
        System.out.println("                                [SKENARIO 2: PENGUJIAN DATA INVALID (TEST INVALID)]

```

```

");
System.out.println("#####");

    System.out.println("\n>>> 2.1 PENGUJIAN NILAI IPK INVALID (TIDAK BOLEH < 0.00 ATAU >
4.00)");
    System.out.println("[UJI COBA 1] Memasukkan IPK Negatif: mhs1.setIpk(-1.50)");
    mhs1.setIpk(-1.50);
    System.out.printf("--> Nilai IPK mhs1 saat ini (harus tetap utuh): %.2f\n", mhs1.getIpk());

    System.out.println("\n[UJI COBA 2] Memasukkan IPK Melebihi Batas: mhs1.setIpk(4.85)");
    mhs1.setIpk(4.85);
    System.out.printf("--> Nilai IPK mhs1 saat ini (harus tetap utuh): %.2f\n", mhs1.getIpk());

    System.out.println("\n>>> 2.2 PENGUJIAN SEMESTER INVALID (TIDAK BOLEH < 1 ATAU > 14)");
    System.out.println("[UJI COBA 3] Memasukkan Semester 0: mhs2.setSemester(0)");
    mhs2.setSemester(0);
    System.out.println("--> Nilai Semester mhs2 saat ini: " + mhs2.getSemester());

    System.out.println("\n[UJI COBA 4] Memasukkan Semester 15: mhs2.setSemester(15)");
    mhs2.setSemester(15);
    System.out.println("--> Nilai Semester mhs2 saat ini: " + mhs2.getSemester());

    System.out.println("\n>>> 2.3 PENGUJIAN NAMA MAHASISWA INVALID (KOSONG / WHITESPACE)");
    System.out.println("[UJI COBA 5] Memasukkan Nama Kosong: mhs1.setNama(\" \")");
    mhs1.setNama(" ");
    System.out.println("--> Nilai Nama mhs1 saat ini: " + mhs1.getNama());

    System.out.println("\n>>> 2.4 PENGUJIAN KUOTA KELAS INVALID");
    System.out.println("[UJI COBA 6] Memasukkan Kuota Kelas Negatif: mk3.setKuotaKelas(-10)");
    mk3.setKuotaKelas(-10);
    System.out.println("--> Nilai Kuota mk3 saat ini: " + mk3.getKuotaKelas());

    System.out.println("\n[UJI COBA 7] Memasukkan Kuota Lebih Kecil dari Peserta Terdaftar:");
    System.out.println("Peserta terdaftar di " + mk3.getNamaMk() + " saat ini: " +
mk3.getPesertaTerdaftar());
    System.out.println("Mencoba ubah kuota menjadi 0: mk3.setKuotaKelas(0)");
    mk3.setKuotaKelas(0);
    System.out.println("--> Nilai Kuota mk3 saat ini: " + mk3.getKuotaKelas());

    // =====
    // BAGIAN 3: VERIFIKASI AKHIR STATUS DATA PASCA PENGUJIAN
    // =====

System.out.println("\n#####");
    System.out.println("
                                LEMBAR CETAK HASIL AKHIR KRS
");

System.out.println("#####\n");
    krs1.tampilkanKrs();

System.out.println("=====");
    System.out.println(" SEMUA PENGUJIAN VALIDASI ENKAPSULASI P3 BERHASIL DILALUI DENGAN
SUKSES ");

System.out.println("=====");
}
}

```


BAB VI: HASIL PENGUJIAN & ANALISIS EKSEKUSI PROGRAM

6.1 Output Terminal Eksekusi Program (Test Valid & Test Invalid)

Program dikompilasi menggunakan javac dan dijalankan dengan java:

```
PS D:\...\P3_3125522015_Mohamad Zaky Bahtiar Arifianto> javac -d bin src/*.java
PS D:\...\P3_3125522015_Mohamad Zaky Bahtiar Arifianto> java -cp bin Main

=====
      PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK (PBO) - MODUL 3
      Encapsulation, Access Modifier, Getter-Setter, dan Validasi Data
=====
Pengembang   : Mohamad Zaky Bahtiar Arifianto
NRP          : 3125522015
Program Studi: D3 Teknik Informatika
Institusi    : PENS PSDKU Sumenep

#####
                        [SKENARIO 1: PENGUJIAN DATA VALID (TEST VALID)]
#####

>>> 1.1 INSTANSIASI OBJEK DENGAN CONSTRUCTOR TERVALIDASI
[STATUS TEST VALID] Seluruh objek berhasil dibuat melalui constructor.

>>> 1.2 MEMBACA DATA MENGGUNAKAN GETTER (ACCESSOR)
Mhs1 - NRP (getNrp)      : 3125522015
Mhs1 - Nama (getNama)    : Mohamad Zaky Bahtiar Arifianto
Mhs1 - IPK (getIpk)      : 3.82
Mhs1 - Kuota SKS (hitung): 24 SKS
MK1  - Nama MK (getNama) : Pemrograman Berorientasi Obyek (3 SKS)
MK1  - Dosen (getDosen)  : Nirwana Haidar Hari, S.Pd., M.Kom.

>>> 1.3 MENGUBAH DATA MENGGUNAKAN SETTER VALID (MUTATOR)
[HASIL SETTER VALID] IPK mhs1 setelah setIpk(3.92): 3.92
[HASIL SETTER VALID] Dosen MK3 setelah setDosenPengampu: Dr. Indah Susilowati, S.T., M.T.

>>> 1.4 TRANSAKSI PENGISIAN KRS & PENGESAHAN OLEH DOSEN WALI
>> [PERMOHONAN KRS] Mahasiswa Mohamad Zaky Bahtiar Arifianto mendaftar MK: Pemrograman
Berorientasi Obyek (3 SKS)...
[BERHASIL] MK 'Pemrograman Berorientasi Obyek' berhasil ditambahkan ke KRS KRS-2026-001
Beban SKS saat ini: 3 / 24 SKS
>> [PERMOHONAN KRS] Mahasiswa Mohamad Zaky Bahtiar Arifianto mendaftar MK: Praktikum PBO (2
SKS)...
[BERHASIL] MK 'Praktikum PBO' berhasil ditambahkan ke KRS KRS-2026-001
Beban SKS saat ini: 5 / 24 SKS
>> [PERMOHONAN KRS] Mahasiswa Mohamad Zaky Bahtiar Arifianto mendaftar MK: Basis Data Lanjut (3
SKS)...
[BERHASIL] MK 'Basis Data Lanjut' berhasil ditambahkan ke KRS KRS-2026-001
Beban SKS saat ini: 8 / 24 SKS
>> [PERMOHONAN KRS] Mahasiswa Mohamad Zaky Bahtiar Arifianto mendaftar MK: Rekayasa Perangkat
Lunak (3 SKS)...
[BERHASIL] MK 'Rekayasa Perangkat Lunak' berhasil ditambahkan ke KRS KRS-2026-001
Beban SKS saat ini: 11 / 24 SKS
>> [PERMOHONAN KRS] Mahasiswa Mohamad Zaky Bahtiar Arifianto mendaftar MK: Jaringan Komputer (3
SKS)...
[BERHASIL] MK 'Jaringan Komputer' berhasil ditambahkan ke KRS KRS-2026-001
```

Beban SKS saat ini: 14 / 24 SKS
[VALIDASI KRS] Dokumen KRS KRS-2026-001 milik Mohamad Zaky Bahtiar Arifianto berhasil DISETUJUI & DISAHKAN oleh Dosen Wali: Nirwana Haidar Hari, S.Pd., M.Kom.
Status Verifikasi KRS 1 (isDisetujui): VALID

[SKENARIO 2: PENGUJIAN DATA INVALID (TEST INVALID)]
#####

>>> 2.1 PENGUJIAN NILAI IPK INVALID (TIDAK BOLEH < 0.00 ATAU > 4.00)
[UJI COBA 1] Memasukkan IPK Negatif: mhs1.setIpk(-1.50)
[VALIDASI DITOLAK] Nilai IPK -1.50 tidak valid! Harus berada di rentang 0.00 s/d 4.00.
--> Nilai IPK mhs1 saat ini (harus tetap utuh): 3.92

[UJI COBA 2] Memasukkan IPK Melebihi Batas: mhs1.setIpk(4.85)
[VALIDASI DITOLAK] Nilai IPK 4.85 tidak valid! Harus berada di rentang 0.00 s/d 4.00.
--> Nilai IPK mhs1 saat ini (harus tetap utuh): 3.92

>>> 2.2 PENGUJIAN SEMESTER INVALID (TIDAK BOLEH < 1 ATAU > 14)
[UJI COBA 3] Memasukkan Semester 0: mhs2.setSemester(0)
[VALIDASI DITOLAK] Semester 0 tidak valid! Batas semester adalah 1 s/d 14.
--> Nilai Semester mhs2 saat ini: 3

[UJI COBA 4] Memasukkan Semester 15: mhs2.setSemester(15)
[VALIDASI DITOLAK] Semester 15 tidak valid! Batas semester adalah 1 s/d 14.
--> Nilai Semester mhs2 saat ini: 3

>>> 2.3 PENGUJIAN NAMA MAHASISWA INVALID (KOSONG / WHITESPACE)
[UJI COBA 5] Memasukkan Nama Kosong: mhs1.setNama(" ")
[VALIDASI DITOLAK] Nama mahasiswa tidak valid! Minimal 3 karakter dan tidak boleh kosong.
--> Nilai Nama mhs1 saat ini: Mohamad Zaky Bahtiar Arifianto

>>> 2.4 PENGUJIAN KUOTA KELAS INVALID
[UJI COBA 6] Memasukkan Kuota Kelas Negatif: mk3.setKuotaKelas(-10)
[VALIDASI DITOLAK] Kuota kelas -10 tidak valid! Kuota harus lebih besar dari 0.
--> Nilai Kuota mk3 saat ini: 2

[UJI COBA 7] Memasukkan Kuota Lebih Kecil dari Peserta Terdaftar:
Peserta terdaftar di Basis Data Lanjut saat ini: 1
Mencoba ubah kuota menjadi 0: mk3.setKuotaKelas(0)
[VALIDASI DITOLAK] Kuota kelas 0 tidak valid! Kuota harus lebih besar dari 0.
--> Nilai Kuota mk3 saat ini: 2

LEMBAR CETAK HASIL AKHIR KRS
#####

=====
KARTU RENCANA STUDI (KRS)
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
=====

Nomor Registrasi KRS : KRS-2026-001
Tahun Ajaran / Sem. : 2026/2027 Ganjil (Semester 3)
Mahasiswa (NRP>Nama) : 3125522015 - Mohamad Zaky Bahtiar Arifianto
Program Studi : D3 Teknik Informatika
Capaian IPK Lalu : 3.92 (Batas Maksimal Beban: 24 SKS)
Status Pengesahan : DISETUJUI (VALID)
Dosen Wali Pengesah : Nirwana Haidar Hari, S.Pd., M.Kom.

NO | KODE | MATA KULIAH | SKS | DOSEN PENGAMPU

1	IF201	Pemrograman Berorientasi Obyek	3	Nirwana Haidar Hari, S.Pd., M.Kom.
2	IF202	Praktikum PBO	2	Nirwana Haidar Hari, S.Pd., M.Kom.
3	IF203	Basis Data Lanjut	3	Dr. Indah Susilowati, S.T., M.T.
4	IF204	Rekayasa Perangkat Lunak	3	Budi Raharjo, S.Kom., M.Kom.
5	IF205	Jaringan Komputer	3	Hendra Kusuma, S.Kom., M.T.

TOTAL SKS TERDAFTAR : 14 SKS

SEMUA PENGUJIAN VALIDASI ENKAPSULASI P3 BERHASIL DILALUI DENGAN SUKSES

6.2 Pembahasan & Analisis Pengujian Data Valid vs Invalid

Dari hasil pengujian terminal di atas dapat ditarik analisis penting:

- 1. Test Valid Terverifikasi: Ketika data yang dimasukkan memenuhi aturan bisnis (IPK 3.92, semester valid, kuota mencukupi), method setter berhasil memperbarui nilai atribut internal objek.
- 2. Test Invalid Berhasil Dibendung: Saat program mencoba memasukkan nilai ilegal (IPK -1.50, IPK 4.85, semester 0 atau 15, nama spasi kosong, dan kuota negatif), method setter menolak perubahan tersebut, mengeluarkan pesan peringatan, dan MEMPERTAHANKAN nilai atribut sebelumnya tanpa terjadi korupsi data.
- 3. Bukti Keberhasilan Encapsulation: Tidak ada jalan pintas bagi pemanggil eksternal untuk merusak state data objek karena seluruh atribut telah terkunci rapat dalam access modifier private.

BAB VII: EVALUASI SPRINT & DEFINITION OF DONE (DoD)

7.1 Sprint Review

Tabel evaluasi capaian implementasi Modul 3:

Item Evaluasi	Hasil Capaian Modul 3
Attribute berhasil di-private	Terpenuhi 100% (Semua atribut Mahasiswa, MataKuliah, dan KRS di-private).
Getter berjalan	Terpenuhi (Method getter membaca data tanpa celah modifikasi).
Setter berjalan	Terpenuhi (Setter selektif memfilter data dengan logika validasi).
Validasi berhasil	Terpenuhi (Validasi IPK, semester, kuota kelas, nama mahasiswa berjalan baik).
Constructor diperbaiki	Terpenuhi (Constructor memanfaatkan validasi saat inisialisasi awal).

Test invalid berhasil	Terpenuhi (Nilai invalid ditolak dan nilai atribut asli tetap utuh).
Program dapat dijalankan	Terpenuhi (Kompilasi javac dan eksekusi runtime berjalan tanpa error).

7.2 Sprint Retrospective

- 1. What Went Well? (Apa yang berhasil dilakukan dengan baik?)
Penerapan konsep enkapsulasi berhasil secara drastis meningkatkan keandalan sistem akademik. Nilai-nilai berbahaya seperti IPK negatif atau kuota minus langsung diblokir di pintu masuk setter dan constructor.
- 2. What Went Wrong? (Kendala saat refactoring?)
Perubahan atribut menjadi private memicu banyak kompilasi error awal pada method antar-class yang sebelumnya mengakses atribut publik secara langsung. Hal ini diselesaikan secara rapi dengan mengganti semua pemanggilan menjadi method getter resmi.
- 3. Improvement (Rencana perbaikan berikutnya?)
Memperdalam relasi struktural antar-objek pada Sprint P4 (Association, Aggregation, dan Composition) agar relasi kepemilikan antar-entitas semakin matang.

7.3 Evaluasi Kesesuaian Definition of Done (DoD) Modul 3

Kriteria Definition of Done (DoD)	Status	Keterangan Evaluasi
Menggunakan project P2	TERPENUHI	Melanjutkan dan merefactor source code dari praktikum P2.
Minimal 3 class direfactor	TERPENUHI	Tiga class domain (Mahasiswa, MataKuliah, KRS) direfactor penuh.
Attribute utama menggunakan private	TERPENUHI	100% atribut dideklarasikan dengan access modifier private.
Getter dibuat sesuai kebutuhan	TERPENUHI	Getter tersedia untuk setiap atribut yang perlu diakses publik.
Setter hanya dibuat bila diperlukan	TERPENUHI	Atribut unik seperti nrp, kodeMk, sks bersifat read-only.
Minimal 3 validasi data diterapkan	TERPENUHI	Diterapkan validasi IPK, semester, kuota, nama, dan batas kuota SKS.
Constructor mengikuti aturan validasi	TERPENUHI	Constructor memanggil setter/validasi internal sebelum inisialisasi.

Tidak ada modifikasi langsung dari Main	TERPENUHI	Seluruh manipulasi state dari Main.java melalui setter resmi.
Terdapat pengujian data valid	TERPENUHI	Diuji pada Skenario 1 (Test Valid).
Terdapat pengujian data tidak valid	TERPENUHI	Diuji pada Skenario 2 (Test Invalid 1 s/d 7).
Program berhasil dikompilasi	TERPENUHI	javac -d bin src/*.java selesai tanpa error.
Program berhasil dijalankan	TERPENUHI	java -cp bin Main tereksekusi sempurna.
Sprint Backlog diperbarui	TERPENUHI	Seluruh item backlog berstatus DONE.
Sprint Review tersedia	TERPENUHI	Tabel evaluasi review telah disajikan lengkap.
Sprint Retrospective tersedia	TERPENUHI	Refleksi pengerjaan terdokumentasi lengkap.

BAB VIII: KESIMPULAN & TARGET SPRINT BERIKUTNYA (P4)

Praktikum Modul 3 Pemrograman Berorientasi Obyek telah berhasil diselesaikan dengan baik dan memenuhi seluruh kriteria kelulusan Definition of Done (DoD). Mahasiswa telah berhasil mengimplementasikan pilar Encapsulation, access modifier private, pembuatan antarmuka getter-setter yang terkontrol, serta integrasi filter validasi data untuk membentengi integritas state objek.

Pada praktikum berikutnya (Pertemuan 4 / P4), fokus pengembangan akan diarahkan pada Object Relationship:

- 1. Memodelkan relasi formal Association, Aggregation, dan Composition antar-entitas SIAKAD.
- 2. Mengatur dependensi siklus hidup (lifecycle) objek anak terhadap objek induk.
- 3. Menyempurnakan interaksi sistem informasi akademik agar lebih tangguh dan siap pakai.